INFORMÁTICA GRÁFICA Y VISUALIZACIÓN

Relación 3

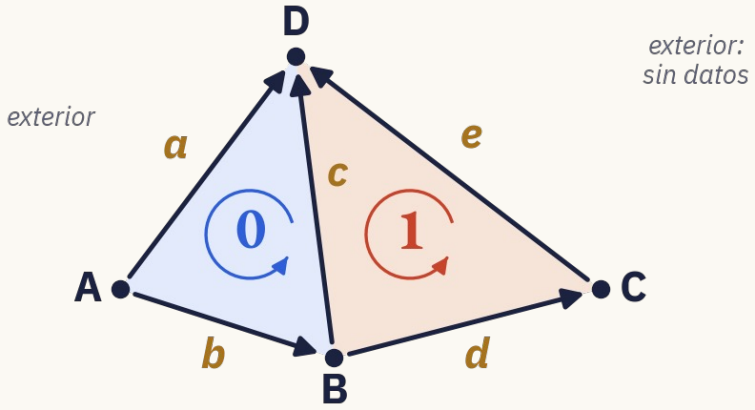
Representación de modelos 3D e interacción

Grado en Ingeniería Informática · Universidad de Jaén

EJERCICIO 1. Aristas aladas de una porción de malla

ENUNCIADO

Dada la porción de una malla de triángulos compuesta por los dos triángulos que se muestra en la figura, represéntela mediante la estructura de datos de arista alada (tabla de aristas, tabla de caras y tabla de vértices), teniendo en cuenta que debe dejar sin completar aquella parte de la estructura para la cual no tiene datos. En la figura, los vértices se identifican con letras mayúsculas, las aristas con letras minúsculas y las caras con números.



CONVENIO (TEMA 4)

Arista orientada vo → vd. Fi: cara que la recorre vo → vd en su contorno CCW; Fd: la otra.

anti / sigi: anterior y siguiente en el contorno de Fi; antd / sigd: ídem en Fd.

TABLA DE ARISTAS

e	vo	vd	Fi	Fd	anti	sigi	antd	sigd
a	A	D	—	0	—	—	*c*	*b*
b	A	B	0	—	*a*	*c*	—	—
c	B	D	0	1	*b*	*a*	*e*	*d*
d	B	C	1	—	*c*	*e*	—	—
e	C	D	1	—	*d*	*c*	—	—

Vértice	Coordenadas	Arista
A	Ax, Ay, Az	*b*
B	Bx, By, Bz	*c*
C	Cx, Cy, Cz	*d*
D	Dx, Dy, Dz	*c*

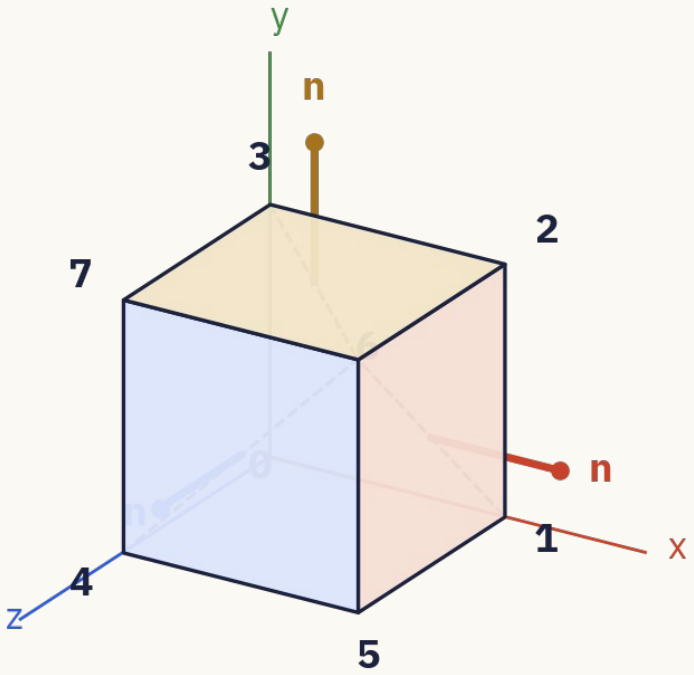
Cara	Aristas	Contorno CCW
0	*a b c*	A → B → D
1	*d e c*	B → C → D

Casillas en blanco (—): Fi de a y Fd de b, d, e (lado exterior, cara desconocida), sus aristas «ala» de ese lado y las coordenadas de los vértices.

EJERCICIO 2. Array de vértices de tres caras de un cubo

ENUNCIADO

Dado un cubo unidad con esquinas en los vértices (0, 0, 0) y (1, 1, 1) represente tres caras conectadas usando la estructura de datos de «vertex array» o array de vértices. Identifique la geometría y la topología. La definición de las caras debe estar preparada para aplicar la técnica de «back face culling». Justifique su solución.



GEOMETRÍA

i	x	y	z
0	0	0	0
1	1	0	0
2	1	1	0
3	0	1	0
4	0	0	1
5	1	0	1
6	1	1	1
7	0	1	1

TOPOLOGÍA (TRIÁNGULOS CCW)

Cara	Triángulos	Normal (v1−v0)×(v2−v0)
z = 1	4, 5, 7 · 5, 6, 7	(0, 0, 1)
x = 1	1, 2, 6 · 1, 6, 5	(1, 0, 0)
y = 1	3, 7, 6 · 3, 6, 2	(0, 1, 0)

JUSTIFICACIÓN

OpenGL (mano derecha): cara frontal = vértices en sentido antihorario vistos desde fuera, es decir, normal hacia el exterior.

Las 6 normales salen del cubo y cada arista compartida (5-6, 2-6, 6-7 y las diagonales 4-6, 1-6, 3-6) se recorre en sentidos opuestos: orientación coherente.

```
glEnable(GL_CULL_FACE); glCullFace(GL_BACK);
glFrontFace(GL_CCW);
glDrawElements(GL_TRIANGLES, 18, GL_UNSIGNED_BYTE, idx);
```

Geometría: 8 vértices (24 reales). Topología: 6 triángulos = 18 índices. Vale cualquier permutación cíclica de cada terna; invertir el orden no.

EJERCICIO 3. Aristas aladas · a) tetraedro

ENUNCIADO

Represente la información topológica y geométrica de las siguientes figuras mediante Aristas Aladas. Indique sobre la figura cada uno de los nombres dados a vértices, aristas y caras. a. De un tetraedro: vértices en (0,1,0), (-1,0,-1), (1,0,-1) y (0,0,1). b. De una pirámide: vértices en (0,0,0), (1,0,0), (1,0,1), (0,0,1) y (0.5, 1, 0.5).

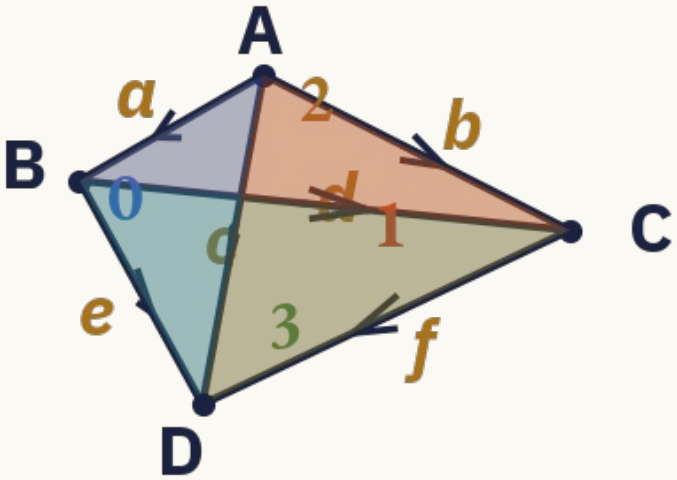


TABLA DE ARISTAS

e	vo	vd	Fi	Fd	anti	sigi	antd	sigd
a	A	B	0	2	*c*	*e*	*d*	*b*
b	A	C	2	1	*a*	*d*	*f*	*c*
c	A	D	1	0	*b*	*f*	*e*	*a*
d	B	C	3	2	*e*	*f*	*b*	*a*
e	B	D	0	3	*a*	*c*	*f*	*d*
f	C	D	3	1	*d*	*e*	*c*	*b*

Vértice	Coordenadas	Arista
A	(0, 1, 0)	*a*
B	(−1, 0, −1)	*d*
C	(1, 0, −1)	*f*
D	(0, 0, 1)	*c*

Cara	Contorno CCW	Aristas
0	A → B → D	*a e c*
1	A → D → C	*f b c*
2	A → C → B	*d b a*
3	B → C → D (base)	*d f e*

Comprobado

Normales exteriores: 0 → (−2, 1, 1) · 1 → (2, 1, 1) · 2 → (0, 2, −2) · 3 → (0, −4, 0). Malla cerrada: ninguna casilla vacía; Euler 4 − 6 + 4 = 2.

EJERCICIO 3. Aristas aladas · b) pirámide de base cuadrada

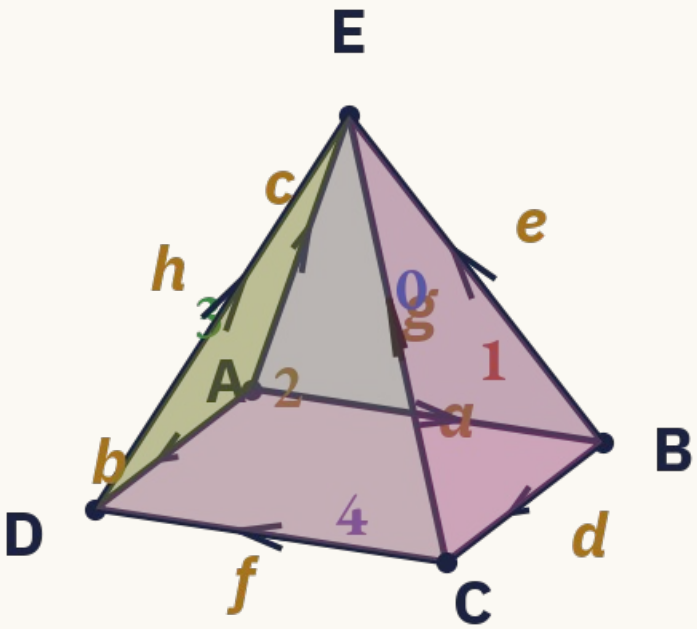


TABLA DE ARISTAS

e	vo	vd	Fi	Fd	anti	sigi	antd	sigd
a	A	B	4	0	*b*	*d*	*e*	*c*
b	A	D	3	4	*c*	*h*	*f*	*a*
c	A	E	0	3	*a*	*e*	*h*	*b*
d	B	C	4	1	*a*	*f*	*g*	*e*
e	B	E	1	0	*d*	*g*	*c*	*a*
f	C	D	4	2	*d*	*b*	*h*	*g*
g	C	E	2	1	*f*	*h*	*e*	*d*
h	D	E	3	2	*b*	*c*	*g*	*f*

Vértice	Coordenadas	Arista
A	(0, 0, 0)	*a*
B	(1, 0, 0)	*d*
C	(1, 0, 1)	*f*
D	(0, 0, 1)	*h*
E	(0,5; 1; 0,5)	*c*

Cara	Contorno CCW	Aristas
0	A → E → B	*c e a*
1	B → E → C	*e g d*
2	C → E → D	*b f g*
3	D → E → A	*c b h*
4	A → B → C → D (base)	*b a d f*

Comprobado

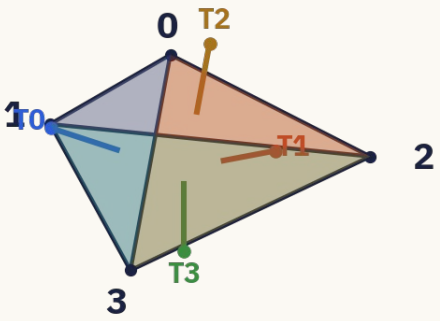
Normales: 0 → (0; 0,5; −1) · 1 → (1; 0,5; 0) · 2 → (0; 0,5; 1) · 3 → (−1; 0,5; 0) · 4 → (0, −1, 0), todas hacia fuera. Euler 5 − 8 + 5 = 2.
La base A, B, C, D (en el orden del enunciado) ya es antihoraria vista desde abajo: su normal apunta a −y.

EJERCICIO 4. Mallas de triángulos indexadas preparadas para back-face culling

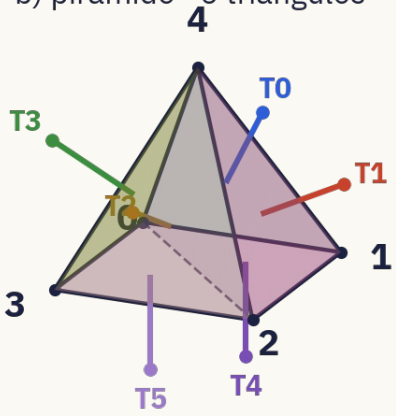
ENUNCIADO

Represente las figuras del ejercicio anterior mediante una estructura que represente un array de vértices y un array de triángulos (representación de mallas de triángulos indexadas). Pase estas estructuras de datos representadas gráficamente a instrucciones de OpenGL que manejen los arrays anteriores. En todos los casos, las estructuras de datos deben estar preparadas para realizar back face culling.

a) tetraedro · 4 triángulos



b) pirámide · 6 triángulos



	Vértices	Triángulos (CCW)	Normal
a	V0 (0, 1, 0)	T0: 0, 1, 3	(−2; 1; 1)
	V1 (−1, 0, −1)	T1: 0, 3, 2	(2; 1; 1)
	V2 (1, 0, −1)	T2: 0, 2, 1	(0; 2; −2)
	V3 (0, 0, 1)	T3: 1, 2, 3	(0; −4; 0)

	Vértices	Triángulos (CCW)	Normal
b	V0 (0, 0, 0)	T0: 0, 4, 1	(0; 0,5; −1)
	V1 (1, 0, 0)	T1: 1, 4, 2	(1; 0,5; 0)
	V2 (1, 0, 1)	T2: 2, 4, 3	(0; 0,5; 1)
	V3 (0, 0, 1)	T3: 3, 4, 0	(−1; 0,5; 0)
	V4 (0,5; 1; 0,5)	T4: 0, 1, 2	(0; −1; 0)
		T5: 0, 2, 3	(0; −1; 0)

CRITERIO

Cada terna, en sentido antihorario visto desde fuera: (v1 − v0) × (v2 − v0) apunta al exterior. La base cuadrada se parte por la diagonal 0-2.

Tetraedro: 4 vértices y 12 índices · Pirámide: 5 vértices y 18 índices. Todas las normales comprobadas hacia fuera.

EJERCICIO 4 · código. Instrucciones OpenGL con vertex arrays

TETRAEDRO

```
// a) Tetraedro
GLfloat vTetra[4][3] = { {0,1,0}, {-1,0,-1},
                         {1,0,-1}, {0,0,1} };
GLubyte iTetra[12] = { 0,1,3,  0,3,2,
                       0,2,1,  1,2,3 };

glEnable(GL_CULL_FACE);
glCullFace(GL_BACK);
glFrontFace(GL_CCW);      // por defecto

glEnableClientState(GL_VERTEX_ARRAY);
glVertexPointer(3, GL_FLOAT, 0, vTetra);
glDrawElements(GL_TRIANGLES, 12,
               GL_UNSIGNED_BYTE, iTetra);
glDisableClientState(GL_VERTEX_ARRAY);
```

PIRÁMIDE

```
// b) Pirámide
GLfloat vPir[5][3] = { {0,0,0}, {1,0,0}, {1,0,1},
                       {0,0,1}, {0.5,1,0.5} };
GLubyte iPir[18] = { 0,4,1,  1,4,2,  2,4,3,
                     3,4,0,  0,1,2,  0,2,3 };

// mismo estado de culling

glEnableClientState(GL_VERTEX_ARRAY);
glVertexPointer(3, GL_FLOAT, 0, vPir);
glDrawElements(GL_TRIANGLES, 18,
               GL_UNSIGNED_BYTE, iPir);
glDisableClientState(GL_VERTEX_ARRAY);
```

El 2.º argumento de glDrawElements es el número de índices (3 × triángulos): 12 y 18. GLubyte basta con menos de 256 vértices; el orden de cada terna decide qué caras sobreviven al culling.

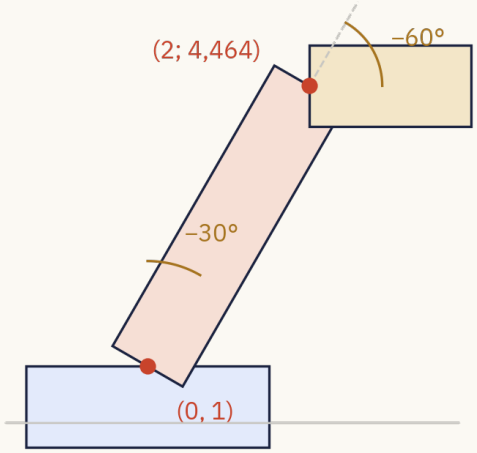
EJERCICIO 5. De la tabla de instanciación al grafo de escena

ENUNCIADO

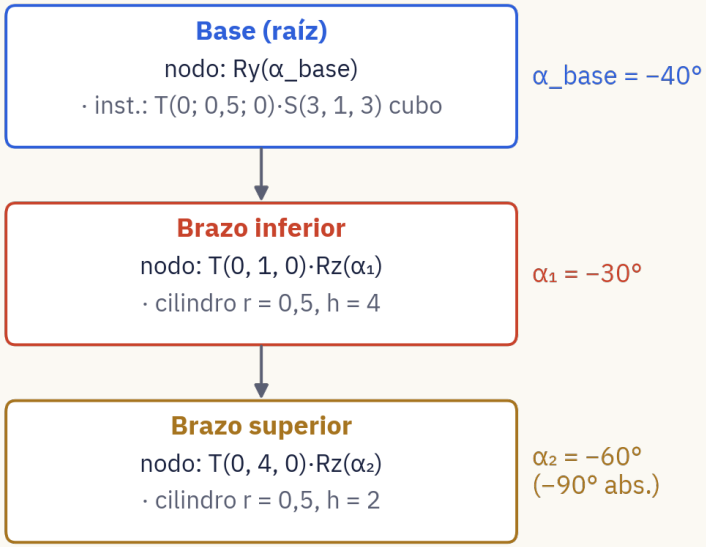
Pase la tabla de instanciación de las transparencias vistas en clase a un grafo de escena, independizando las transformaciones geométricas necesarias. Puede también pasarlo a código OpenGL usando la pila de matrices.

TABLA DE INSTANCIACIÓN (TEMA 4, ORDEN DE APLICACIÓN)

Símbolo	Parámetros	Transformaciones de instanciación
1 (cubo)	3 × 1 × 3	tras(0; 0,5; 0) · rot(−40°, y)
2 (cilindro)	r = 0,5; h = 4	rot(−30°, z) · tras(0; 1; 0) · rot(−40°, y)
2 (cilindro)	r = 0,5; h = 2	rot(−90°, z) · tras(2,000; 4,464; 0) · rot(−40°, y)



vista frontal, α_base = 0 (plano xy)



INDEPENDIZAR

rot(−40°, y) aparece en las tres filas: va una sola vez, en la raíz.

tras(2; 4,464) = codo calculado a mano: se sustituye por T(0, 4, 0) en el sistema del brazo inferior.

rot(−90°) absoluto = −30° heredado + **−60°** propio.

Grafo: 3 nodos en cadena, 3 grados de libertad (α_base, α_1, α_2).

EJERCICIO 5 · código. El grafo con la pila de matrices

CÓDIGO OPENGL

```
glRotatef(ANG_BASE, 0, 1, 0);            // raíz: α_base = −40

glPushMatrix();                          // instancia de la base
  glTranslatef(0, 0.5, 0);
  glScalef(3, 1, 3);
  cubo();                                // cubo unidad centrado
glPopMatrix();

glPushMatrix();                          // brazo inferior
  glTranslatef(0, 1, 0);
  glRotatef(ANG_INF, 0, 0, 1);           // α₁ = −30
  cilindro(0.5, 4);                     // base en y = 0
  glPushMatrix();                        // brazo superior
    glTranslatef(0, 4, 0);
    glRotatef(ANG_SUP, 0, 0, 1);         // α₂ = −60
    cilindro(0.5, 2);
  glPopMatrix();
glPopMatrix();
```

COMPROBACIÓN

Ry(−40)·T(0,1,0)·Rz(−30) = fila 2 de la tabla.

Ry(−40)·T(0,1,0)·Rz(−30)·T(0,4,0)·Rz(−60) = fila 3.

Extremo del brazo inferior = base del superior = (1,532; 4,464; 1,286) en el mundo.

Pila vacía al terminar: 3 push y 3 pop.

ERROR TÍPICO

Copiar en cada nodo la fila completa de la tabla: el hijo hereda lo del padre y rot(−40°, y), la traslación 0,5 y los giros se cuentan dos veces (la base del brazo inferior saldría en (0; 1,5; 0)).

Resultado

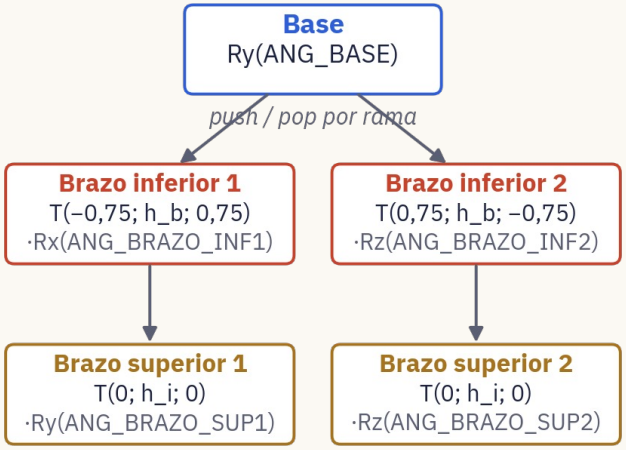
Cada nodo lleva solo su transformación respecto al padre; la escala de la base queda aislada en su push/pop.

EJERCICIO 6. Push y pop en el recorrido del robot de dos brazos

ENUNCIADO

Siga las reglas indicadas en las transparencias vistas en clase referidas al uso de la pila de matrices para implementar el código OpenGL que recorre el grafo de escena para el árbol dado en dichas transparencias, de manera que el código que ya hay (optimizado) incluya todas las llamadas a glPushMatrix() y glPopMatrix() del recorrido del árbol o grafo de escena de ejemplo. Puede probar en OpenGL dicho código para comprobar su funcionamiento.

```
glRotatef(ANG_BASE, 0, 1, 0);            // raíz: base
base();
glPushMatrix();                          // rama 1
  glTranslatef(-0.75, ALTURA_BASE, 0.75);
  glRotatef(ANG_BRAZO_INF1, 1, 0, 0);
  brazo_inferior();
  glPushMatrix();                        // brazo superior 1
    glTranslatef(0, ALTURA_BRAZO_INFERIOR, 0);
    glRotatef(ANG_BRAZO_SUP1, 0, 1, 0);
    brazo_superior();
  glPopMatrix();
glPopMatrix();                           // vuelve a la base
glPushMatrix();                          // rama 2
  glTranslatef(0.75, ALTURA_BASE, -0.75);
  glRotatef(ANG_BRAZO_INF2, 0, 0, 1);
  brazo_inferior();
  glPushMatrix();                        // brazo superior 2
    glTranslatef(0, ALTURA_BRAZO_INFERIOR, 0);
    glRotatef(ANG_BRAZO_SUP2, 0, 0, 1);
    brazo_superior();
  glPopMatrix();
glPopMatrix();
```



COMPROBACIÓN

Con ANG_BASE = 30, alturas 1 y 3 y ángulos 20, 40, −25, 35: orígenes inf1 (−0,275; 1; 1,025), sup1 (0,239; 3,819; 1,913), inf2 (0,275; 1; −1,025), sup2 (1,373; 3,719; −1,658).

4 push y 4 pop; profundidad máx. 2; pila vacía al final.

EJERCICIO 7. Qué guarda un nodo y en qué orden se aplican las transformaciones

ENUNCIADO

Dado un grafo de escena, indique qué información se incluye en cada nodo, qué representa y en qué orden deben ponerse las transformaciones geométricas de cada primitiva. Indique también el orden en el que deben aplicarse las transformaciones de forma jerárquica. Finalmente considere cómo puede parametrizarse cada transformación para incluir los grados de libertad del movimiento.



Composición (código, de la raíz a la hoja):

$$M_{pieza} = M_{raíz} \cdot \ldots \cdot M_{padre} \cdot M_{nodo} \cdot M_{instancia}$$

Sobre los vértices actúa primero la instancia (S, luego R, luego T) y al final la raíz.

1 · INFORMACIÓN DEL NODO

Una parte del modelo con su sistema local: transformación de nodo respecto al padre (se hereda), primitiva opcional con su transformación de instancia (no se hereda) y punteros a los hijos.

2 · ORDEN DENTRO DEL NODO

Sobre la primitiva: primero instancia (S → R → T) y después nodo. En OpenGL (post-multiplica) se escribe al revés: nodo, luego glTranslate, glRotate, glScale y la primitiva.

3 · ORDEN JERÁRQUICO

Preorden desde la raíz: M_hijo = M_padre · M_local. Push al bajar a un hijo, pop al volver.

4 · GRADOS DE LIBERTAD

Parámetros en la transformación de NODO: θ en rotaciones (pivote en el origen del nodo o T(p)·R(θ)·T(−p)), t en traslaciones, con sus rangos. Mueven todo el subárbol.

EJERCICIO 8. Algoritmo genérico de recorrido de un grafo de escena

ENUNCIADO

Diseñe un algoritmo para recorrer un grafo de escena y realizar las llamadas correspondientes para visualizar un modelo representado con dicho grafo de escena de manera genérica. Implemente dicho algoritmo en OpenGL y utilícelo para visualizar un grafo de escena.

ALGORITMO (PREORDEN)

```
recorre(nodo):
  si nodo = NULO: retorna
  PushMatrix()              // guarda la CTM del padre
  MultMatrix(nodo.M)        // CTM = CTM_padre · M_nodo
  si nodo.primitiva: dibújala
  para cada hijo h: recorre(h)
  PopMatrix()               // restaura la CTM del padre
```

PROPIEDADES

Coste O(n.º de nodos visitados); profundidad de pila = profundidad del grafo (GL_MODELVIEW garantiza ≥ 32 niveles).

Árbol HI-HerD: número arbitrario de hijos con dos punteros.

Nodo compartido (grafo acíclico): se visita una vez por camino, con la CTM de ese camino.

Animar = cambiar los parámetros y recalcular la matriz local antes del recorrido.

IMPLEMENTACIÓN EN C / OPENGL

```
typedef struct Nodo {
  GLfloat m[16];            // matriz local (por columnas)
  void (*dibuja)(void);     // primitiva o NULL
  struct Nodo *hijo;        // primer hijo
  struct Nodo *hermano;     // siguiente hermano
} Nodo;

void recorre(const Nodo *n) {
  if (n == NULL) return;
  glPushMatrix();
  glMultMatrixf(n->m);
  if (n->dibuja) n->dibuja();
  for (const Nodo *h = n->hijo; h; h = h->hermano)
    recorre(h);
  glPopMatrix();
}
```

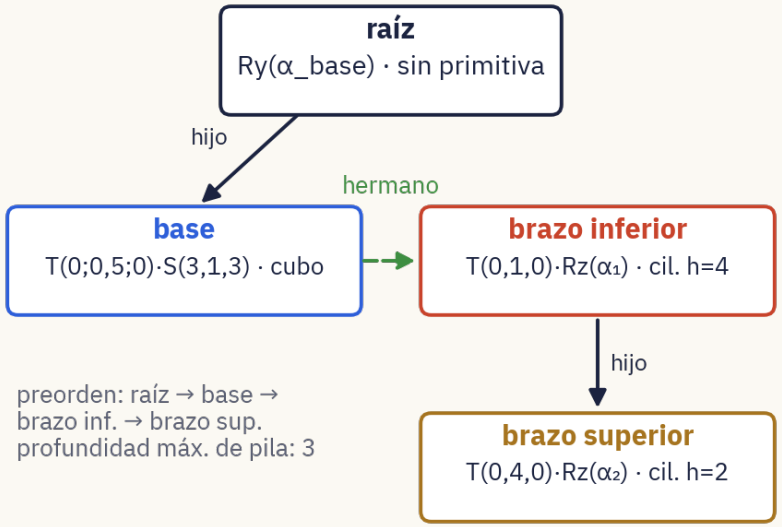
EJERCICIO 8 · ejemplo. Uso del algoritmo con el robot del ejercicio 5

CONSTRUCCIÓN Y USO

```
Nodo *nuevo(void (*d)(void)) {        // matriz identidad
  Nodo *n = calloc(1, sizeof(Nodo)); n->dibuja = d;
  glPushMatrix(); glLoadIdentity();
  glGetFloatv(GL_MODELVIEW_MATRIX, n->m); glPopMatrix();
  return n;
}
void fijaMatriz(Nodo *n, void (*transf)(void)) {
  glPushMatrix(); glLoadIdentity(); transf();
  glGetFloatv(GL_MODELVIEW_MATRIX, n->m); glPopMatrix();
}
void anade(Nodo *p, Nodo *h) {            // al final de la lista
  Nodo **q = &p->hijo; while (*q) q = &(*q)->hermano; *q = h;
}

/* construcción */
raiz = nuevo(NULL);     fijaMatriz(raiz, T_raiz);  // Ry(a_base)
base = nuevo(cubo);     fijaMatriz(base, T_base);  // T(0,.5,0) S(3,1,3)
inf  = nuevo(cil4);     fijaMatriz(inf,  T_inf);   // T(0,1,0) Rz(a1)
sup  = nuevo(cil2);     fijaMatriz(sup,  T_sup);   // T(0,4,0) Rz(a2)
anade(raiz, base); anade(raiz, inf); anade(inf, sup);

/* en display() */  recorre(raiz);
```



COMPROBACIÓN

Orden de dibujo: base → brazo inferior → brazo superior.

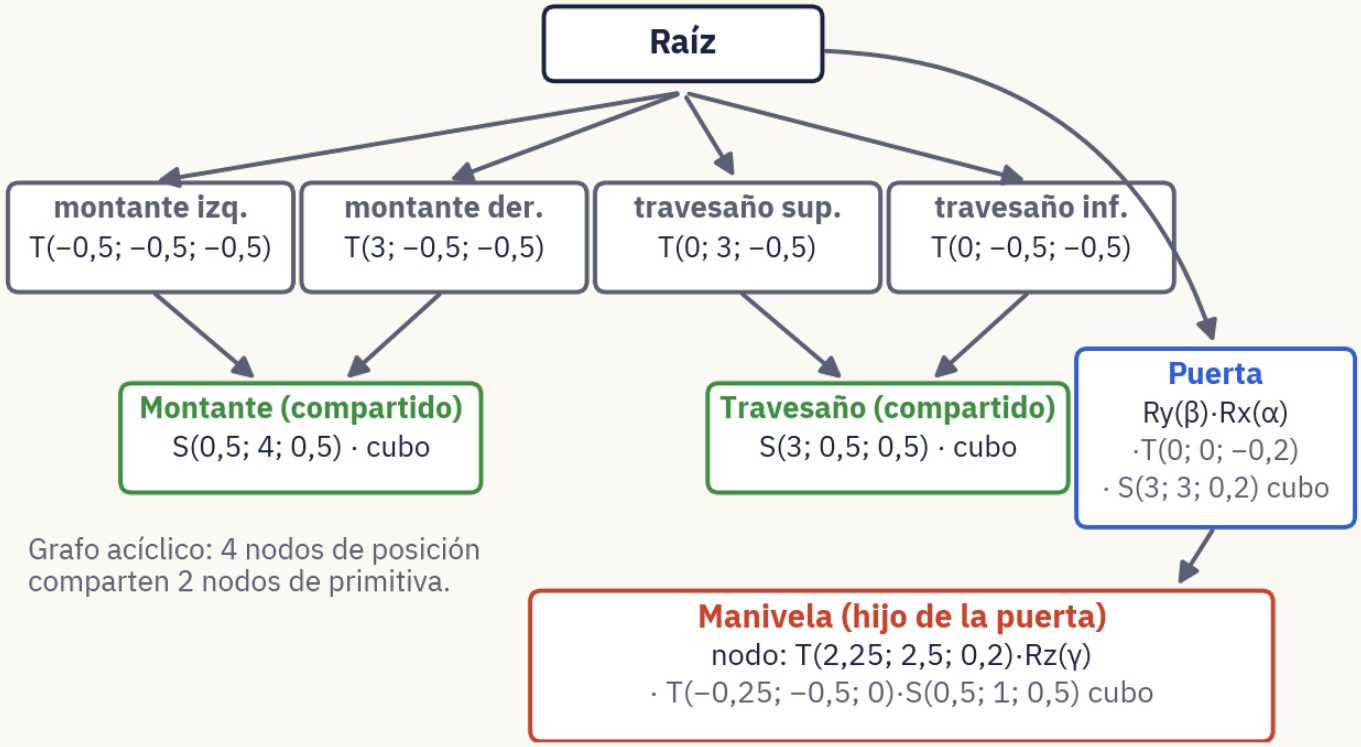
Matrices de las tres primitivas = las de la tabla de instanciación del ejercicio 5 (tolerancia 1e-9).

La escala de la base no llega a su hermano; profundidad máx. 3; pila vacía al final.

EJERCICIO 9. Grafo de escena de una ventana con tres grados de libertad

ENUNCIADO

Elabore un grafo de escena representando el siguiente modelo. Se debe diseñar la estructura y transformaciones del modelo, permitiendo los grados de libertad indicados de manera parametrizada. Hay una única primitiva inicial, un cubo unidad con esquinas opuestas en (0,0,0) y (1,1,1). El modelo debe tener 3 grados de libertad para abrir la puerta desde arriba o desde el lado y rotar la manivela adecuadamente. Debe mostrar los parámetros que habría que dar al modelo para representar los tres escenarios mostrados: Ventana totalmente cerrada, ventana abierta desde arriba y ventana abierta lateralmente.



Grafo acíclico: 4 nodos de posición comparten 2 nodos de primitiva.

DISEÑO

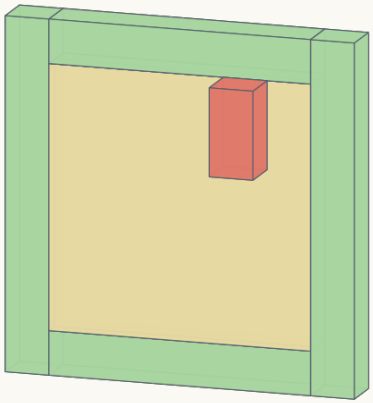
Primitiva única: cubo $[0, 1]^3$.

Marco 4 × 4 de grosor 0,5 con hueco 3 × 3; puerta 3 × 3 × 0,2.

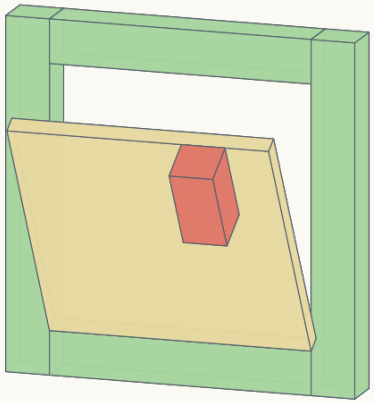
α: giro sobre la arista inferior (eje x); β: giro sobre la arista izquierda (eje y); γ: giro de la manivela (eje z).

EJERCICIO 9 · escenarios. Parámetros de los tres escenarios

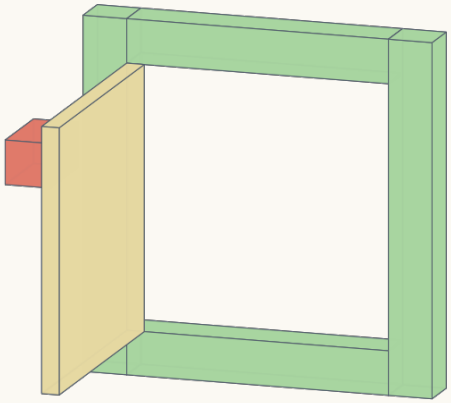
cerrada: α = 0, β = 0, γ = 0



desde arriba: α = 30°, β = 0, γ = 0



lateral: α = 0, β = −90°, γ = 90°



Escenario	α (arriba)	β (lateral)	γ (manivela)	Comprobación
Cerrada	0°	0°	0°	puerta x, y ∈ [0, 3], z ∈ [−0,2; 0]: llena el hueco 3 × 3
Abierta desde arriba	α ∈ [0°, 30°] (p. ej. 30°)	0°	0°	con α = 30° el borde superior va a y = 2,598, z = 1,5 (hacia +z)
Abierta lateralmente	0°	β ∈ [−90°, 0°] (p. ej. −90°)	90°	puerta en x ∈ [0; 0,2], z ∈ [0, 3]; manivela horizontal

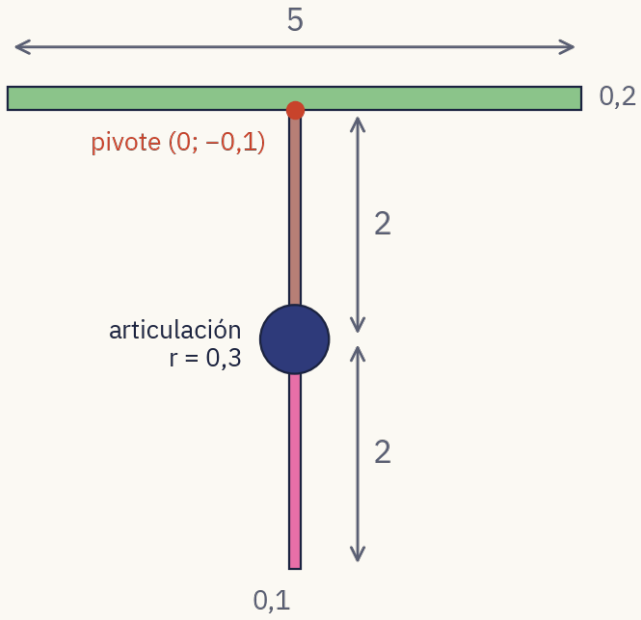
Signos

α > 0 lleva el borde superior hacia el observador (+z); β < 0 abre la hoja hacia +z con la bisagra a la izquierda (β > 0 la llevaría por detrás del marco). γ = 90° pone la manivela horizontal, como en la figura.

EJERCICIO 10. Brazo articulado 2D con el brazo inferior siempre vertical

ENUNCIADO

Indique la combinación de transformaciones necesarias para obtener el modelo final a partir de las primitivas indicadas, con los grados de libertad necesarios, con las siguientes restricciones: – El ángulo máximo de cada grado de libertad es de 45 grados. – El brazo inferior siempre debe estar totalmente vertical. La idea es construir el modelo indicando las transformaciones necesarias, incluyendo los grados de libertad necesarios. Para ello puede utilizar, aunque no es obligatorio, llamadas a la pila de matrices (de tipo Push y Pop), o introducir un modelo jerárquico. Las transformaciones geométricas en 2D pueden ser indicadas con llamadas del tipo T(x,y), S(x,y), R(alfa) e indicar el orden correcto de aplicación de las transformaciones. Incluya todas las explicaciones necesarias para comprender adecuadamente el funcionamiento del movimiento del modelo. Primitivas: Cuadrado de lado unidad, centrado en (0,0). Círculo de radio 0.3, centrado en (0,0).



ANÁLISIS

Barra fija 5 × 0,2 centrada en el origen.

Brazo superior: cuelga del centro de la barra y gira α ∈ [−45°, 45°] sobre su extremo superior.

Articulación: círculo r = 0,3 en el otro extremo (a 2 del pivote).

Brazo inferior: hereda α; para seguir vertical gira −α. No es un grado de libertad independiente.

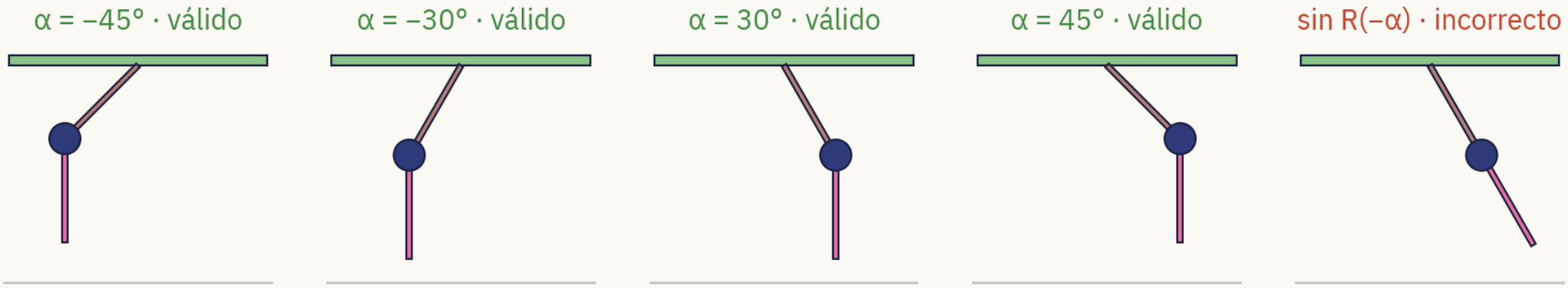
Un solo grado de libertad: α.

Nodo	Transformación de nodo	Instancia + primitiva
Barra	—	S(5; 0,2) · cuadrado
Brazo sup.	T(0; −0,1) · R(α)	T(0, −1) · S(0,1; 2) · cuadrado
Articulación	T(0, −2)	círculo
Brazo inf.	R(−α)	T(0, −1) · S(0,1; 2) · cuadrado

Jerarquía

Barra → Brazo superior → Articulación → Brazo inferior. R(α)·…·R(−α): giro neto 0 en el brazo inferior.

EJERCICIO 10 · código. Pila de matrices y posturas válidas



CÓDIGO

```
Push();  S(5, 0.2);  cuadrado();  Pop();      // barra fija
Push(); T(0, -0.1);  R(alfa);                 // pivote + g.d.l.
 Push();  T(0, -1); S(0.1, 2); cuadrado(); Pop(); // brazo sup.
 T(0, -2);                                    // extremo del brazo
 Push(); circulo(); Pop();                    // articulación
 Push();  R(-alfa);                           // compensa el giro
  T(0, -1); S(0.1, 2); cuadrado();
 Pop();
Pop();                            // alfa ∈ [-45°, 45°]
```

COMPROBACIÓN

α = 0: articulación (0; −2,1), brazo inferior y ∈ [−4,1; −2,1].

α = 30°: articulación (1; −1,832); brazo inferior 0,1 × 2 vertical, x ∈ [0,95; 1,05].

α = ±45°: articulación (±1,414; −1,514), brazo inferior vertical.

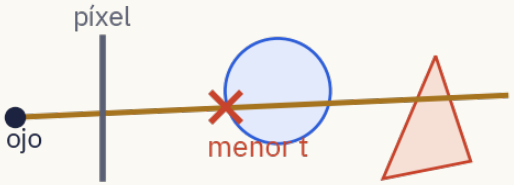
Sin R(−α) el brazo inferior continúa la inclinación del superior: es la postura «incorrecta» de la figura.

EJERCICIO 11. Ventajas, desventajas y comparación de los métodos de selección

ENUNCIADO

Indique las ventajas y desventajas, así como la comparación entre los siguientes métodos de selección: ray-tracing, cajas envolventes y buffer de color.

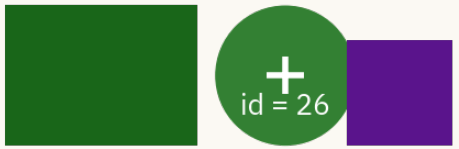
Intersección de rayos



Cajas englobantes (AABB)



Buffer de color



glReadPixels(x, alto − 1 − y) → (51, 128, 51)

	Intersección de rayos	Cajas englobantes (AABB)	Buffer de color
Idea	rayo ojo → píxel; objeto con menor t	rayo o punto contra la caja de cada objeto	color = id; glReadPixels bajo el cursor
Ventajas	precisión geométrica; da el punto 3D y t; no depende del hardware	muy rápido y fácil; ideal como pre-descarte	exacto por píxel; respeta la oclusión; fácil
Desventajas	coste con muchos triángulos; más difícil de programar	falsos positivos (caja > objeto); recalcular al mover	un redibujado extra; límite $2^{24} - 1$ ids; solo lo visible
Precisión	perfecta	baja	perfecta por píxel
Coste	alto en CPU	muy bajo	un redibujado completo

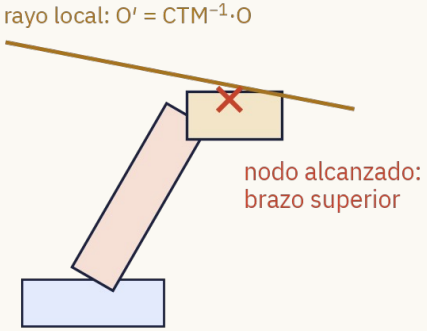
En la práctica se combinan: AABB como fase amplia y rayo exacto o buffer de color como fase fina. El rayo devuelve todos los objetos atravesados; el buffer de color, solo el visible.

EJERCICIO 12. Selección de partes de un objeto articulado

ENUNCIADO

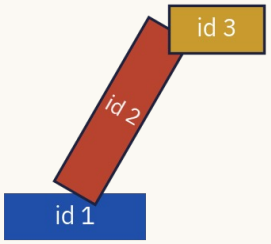
Describa cómo podría aplicarse cada uno de los métodos de selección de objetos vistos en teoría para seleccionar partes de un objeto articulado (representado mediante un grafo de escena), es decir, aplicarlos a un modelo representado de manera jerárquica para seleccionar una parte determinada del modelo en lugar del modelo completo.

Rayo contra cada nodo



Un color (id) por nodo

tabla id → nodo (+ puntero al padre)



INTERSECCIÓN DE RAYOS

Recorrer el grafo acumulando la CTM; llevar el rayo al sistema local de cada primitiva ($O' = CTM^{-1}\cdot O$, $d' = CTM^{-1}\cdot d$) e intersectar con la primitiva sin transformar. Gana el nodo con menor $t > 0$.

CAJAS ENGLOBANTES

Jerarquía de volúmenes: cada nodo guarda la caja de su primitiva y de sus hijos. Se desciende solo por las cajas que corta el rayo (se podan ramas) y se decide en las hojas; recalcular al mover articulaciones.

BUFFER DE COLOR

En el mismo recorrido, antes de dibujar cada primitiva: glColor3ub con su id (id = R + 256·G + 65 536·B); leer el píxel y buscar el nodo en la tabla id → nodo.

Clave: el resultado se asocia a un nodo, no al modelo; con un puntero al padre se sube hasta el nivel que se quiera seleccionar (pieza, brazo o modelo completo).